

```
"""
```

```
MediaFire MCP Server
```

```
=====
```

```
FastMCP server for the MediaFire Core API v2.
```

```
Auth env vars:
```

```
MEDIAFIRE_APP_ID      - MediaFire Application ID
MEDIAFIRE_API_KEY      - MediaFire API Key (from developer portal)
MEDIAFIRE_EMAIL        - MediaFire account email
MEDIAFIRE_PASSWORD     - MediaFire account password
```

```
The server obtains a session token on first use and refreshes it automatically.
```

```
API reference: https://www.mediafire.com/developers/core\_api/1.5/
```

```
"""
```

```
import os
```

```
import json
```

```
import hashlib
```

```
import time
```

```
import httpx
```

```
from typing import Optional, Dict, Any, List
```

```
from pydantic import BaseModel, Field, ConfigDict
```

```
from mcp.server.fastmcp import FastMCP
```

```
# — Constants —————
```

```
APP_ID      = os.environ.get("MEDIAFIRE_APP_ID", "")
API_KEY     = os.environ.get("MEDIAFIRE_API_KEY", "")
EMAIL       = os.environ.get("MEDIAFIRE_EMAIL", "")
PASSWORD    = os.environ.get("MEDIAFIRE_PASSWORD", "")
BASE_URL    = "https://www.mediafire.com/api/1.5"
TIMEOUT     = 60.0
TOKEN_TTL   = 3600 * 8  # 8 hours
```

```
mcp = FastMCP("mediafire_mcp")
```

```
# — Session state —————
```

```
_session_token: str = ""
```

```
_session_ts: float = 0.0
```

```
def _signature(email: str, password: str, app_id: str, api_key: str) -> str:
```

```
    raw = email + password + app_id + api_key
```

```
    return hashlib.md5(raw.encode()).hexdigest()
```

```

async def _get_session() -> str:
    global _session_token, _session_ts
    if _session_token and (time.time() - _session_ts) < TOKEN_TTL:
        return _session_token
    if not all([APP_ID, API_KEY, EMAIL, PASSWORD]):
        raise RuntimeError(
            "MEDIAFIRE_APP_ID, MEDIAFIRE_API_KEY, MEDIAFIRE_EMAIL, MEDIAFIRE_PASS
        )
    sig = _signature(EMAIL, PASSWORD, APP_ID, API_KEY)
    async with httpx.AsyncClient(timeout=TIMEOUT) as client:
        r = await client.get(f"{BASE_URL}/user/get_session_token.php", params={
            "email": EMAIL,
            "password": PASSWORD,
            "app_id": APP_ID,
            "signature": sig,
            "response_format": "json",
        })
        r.raise_for_status()
        data = r.json()["response"]
        if data.get("result") != "Success":
            raise RuntimeError(f"MediaFire auth error: {data.get('message')}")
        _session_token = data["session_token"]
        _session_ts = time.time()
    return _session_token

```

```

async def _api(endpoint: str, params: Optional[Dict] = None,
               method: str = "GET") -> Dict[str, Any]:
    token = await _get_session()
    p = dict(params or {})
    p["session_token"] = token
    p["response_format"] = "json"
    async with httpx.AsyncClient(timeout=TIMEOUT) as client:
        url = f"{BASE_URL}/{endpoint}.php"
        if method == "POST":
            r = await client.post(url, data=p)
        else:
            r = await client.get(url, params=p)
        r.raise_for_status()
        resp = r.json()["response"]
        if resp.get("result") != "Success":
            raise RuntimeError(f"MediaFire API error: {resp.get('message', 'Unkno
        return resp

```

```

def _fmt_size(b: int) -> str:
    for unit in ["B", "KB", "MB", "GB", "TB"]:
        if b < 1024:
            return f"{b:.1f} {unit}"
        b /= 1024
    return f"{b:.1f} PB"

# — Input Models —————
class FolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    folder_key: str = Field(default="myfiles", description="Folder key ('myfiles'
    chunk: int = Field(default=1, description="Page number for pagination", ge=1)
    chunk_size: int = Field(default=100, description="Items per page", ge=1, le=1
    content_type: str = Field(default="both", description="'files', 'folders', or

class FileKeyInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    quick_key: str = Field(..., description="MediaFire file quick_key")

class FolderKeyInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    folder_key: str = Field(..., description="MediaFire folder key")

class CreateFolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    foldername: str = Field(..., description="New folder name", min_length=1, max
    parent_key: str = Field(default="myfiles", description="Parent folder key")

class MoveFileInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    quick_keys: str = Field(..., description="Comma-separated file quick_keys to
    folder_key: str = Field(..., description="Destination folder key")

class CopyFileInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    quick_keys: str = Field(..., description="Comma-separated file quick_keys to
    folder_key: str = Field(..., description="Destination folder key")

class RenameFileInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")

```

```

    quick_key: str = Field(..., description="File quick_key to rename")
    filename: str = Field(..., description="New filename (with extension)")

class SearchInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    search_text: str = Field(..., description="Search term", min_length=1)
    search_all: bool = Field(default=True, description="Search entire account (Tr

class UploadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    local_path: str = Field(..., description="Absolute local path to file")
    folder_key: str = Field(default="myfiles", description="Destination folder ke

class UpdatePrivacyInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    quick_key: str = Field(..., description="File quick_key")
    privacy: str = Field(..., description="'public' or 'private'")

# — Tools —————
@mcp.tool(name="mf_get_user_info",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mf_get_user_info() -> str:
    """Return MediaFire account info: display name, email, storage used/available
    try:
        data = await _api("user/get_info")
        u = data.get("user_info", {})
        used = int(u.get("used_storage_size", 0))
        total = int(u.get("storage_limit", 0))
        lines = [
            "## MediaFire Account Info",
            f"- **Display Name**: {u.get('display_name')}",
            f"- **Email**: {u.get('email')}",
            f"- **Used**: {_fmt_size(used)}",
            f"- **Total**: {_fmt_size(total)}",
            f"- **Free**: {_fmt_size(max(0, total - used))}",
            f"- **Premium**: {u.get('premium', 'no')}",
        ]
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="mf_list_folder",

```

```

        annotations={"readOnlyHint": True, "destructiveHint": False})
async def mf_list_folder(params: FolderInput) -> str:
    """List contents of a MediaFire folder by folder key ('myfiles' = root)."""
    try:
        data = await _api("folder/get_content", {
            "folder_key":    params.folder_key,
            "chunk":         params.chunk,
            "chunk_size":    params.chunk_size,
            "content_type":  params.content_type,
            "order_by":      "name",
        })
        content = data.get("folder_content", {})
        folders = content.get("folders", [])
        files    = content.get("files", [])
        more     = content.get("more_chunks", "no") == "yes"
        lines    = [f"### Folder: `{params.folder_key}` (page {params.chunk})",
                    f"***{len(folders)} folders, {len(files)} files***", ""]
        if folders:
            lines.append("### Folders")
            for f in folders:
                lines.append(f"📁 **{f.get('name')}** (key=`{f.get('folderkey')}`)")
        if files:
            lines.append("\n### Files")
            for f in files:
                size_str = _fmt_size(int(f.get("size", 0))) if f.get("size") else
                lines.append(f"📄 **{f.get('filename')}** {size_str} (key=`{f.g")
        if more:
            lines.append(f"\n_More items on page {params.chunk + 1}_")
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"

```

```

@mcp.tool(name="mf_get_file_info",
        annotations={"readOnlyHint": True, "destructiveHint": False})
async def mf_get_file_info(params: FileKeyInput) -> str:
    """Get metadata for a MediaFire file by its quick_key."""
    try:
        data = await _api("file/get_info", {"quick_key": params.quick_key})
        f = data.get("file_info", {})
        lines = [
            "## File Info",
            f"- **Name**: {f.get('filename')}",
            f"- **Quick Key**: `{f.get('quickkey')}`",
            f"- **Size**: {_fmt_size(int(f.get('size', 0)))}",
            f"- **MIME type**: {f.get('mimetype')}",
            f"- **Privacy**: {f.get('privacy')}",

```

```

        f"- **Created**: {f.get('created')}}",
        f"- **Downloads**: {f.get('downloads')}}",
        f"- **Views**: {f.get('views')}}",
        f"- **MD5**: `{f.get('hash')}`",
    ]
    if f.get("links", {}).get("normal_download"):
        lines.append(f"- **Download link**: {f['links']['normal_download']}")
    return "\n".join(lines)
except Exception as e:
    return f"Error: {e}"

@mcp.tool(name="mf_get_download_link",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mf_get_download_link(params: FileKeyInput) -> str:
    """Get the direct download link for a MediaFire file."""
    try:
        data = await _api("file/get_links", {"quick_key": params.quick_key, "link
links = data.get("links", [{}])
        dl = links[0].get("direct_download", "") if links else ""
        return f"🔗 Direct download: `{dl}`" if dl else "No direct download link
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="mf_create_folder",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mf_create_folder(params: CreateFolderInput) -> str:
    """Create a new MediaFire folder inside a parent folder."""
    try:
        data = await _api("folder/create", {
            "foldername": params.foldername,
            "parent_key": params.parent_key,
        }, method="POST")
        f = data.get("folder_info", {})
        return f"✅ Folder created: **{f.get('name')}** (key=`{f.get('folderkey')}`
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="mf_delete_file",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempoten
async def mf_delete_file(params: FileKeyInput) -> str:
    """Move a MediaFire file to Trash (recoverable)."""
    try:
        await _api("file/delete", {"quick_key": params.quick_key}, method="POST")
        return f"🗑️ File `{params.quick_key}` moved to Trash."

```

```

except Exception as e:
    return f"Error: {e}"

@mcp.tool(name="mf_delete_folder",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempoten
async def mf_delete_folder(params: FolderKeyInput) -> str:
    """Move a MediaFire folder to Trash (recoverable)."""
    try:
        await _api("folder/delete", {"folder_key": params.folder_key}, method="PO
        return f"🗑️ Folder `{params.folder_key}` moved to Trash."
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="mf_move_files",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mf_move_files(params: MoveFileInput) -> str:
    """Move one or more MediaFire files to a destination folder."""
    try:
        await _api("file/move", {"quick_key": params.quick_keys, "folder_key": pa
        return f"📁 Files moved to folder `{params.folder_key}`."
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="mf_copy_files",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mf_copy_files(params: CopyFileInput) -> str:
    """Copy one or more MediaFire files to a destination folder."""
    try:
        await _api("file/copy", {"quick_key": params.quick_keys, "folder_key": pa
        return f"📄 Files copied to folder `{params.folder_key}`."
    except Exception as e:
        return f"Error: {e}"

@mcp.tool(name="mf_rename_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mf_rename_file(params: RenameFileInput) -> str:
    """Rename a MediaFire file."""
    try:
        await _api("file/update", {"quick_key": params.quick_key, "filename": par
        return f"✏️ File renamed to **{params.filename}**."
    except Exception as e:
        return f"Error: {e}"

```

```

@mcp.tool(name="mf_search",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mf_search(params: SearchInput) -> str:
    """Search MediaFire for files and folders by name."""
    try:
        p: Dict[str, Any] = {"search_text": params.search_text}
        if params.search_all:
            p["filter"] = "all"
        data = await _api("file/search", p)
        results = data.get("file_result_list", [])
        if not results:
            return f"No results for '{params.search_text}'."
        lines = [f"## Search: '{params.search_text}'", f"**{len(results)} found**"]
        for r in results[:50]:
            size_str = _fmt_size(int(r.get("size", 0))) if r.get("size") else "?"
            lines.append(f"

```

```

@mcp.tool(name="mf_update_privacy",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote": True})
async def mf_update_privacy(params: UpdatePrivacyInput) -> str:
    """Set a MediaFire file's privacy to 'public' or 'private'."""
    try:
        await _api("file/update", {"quick_key": params.quick_key, "privacy": params.privacy})
        return f"
```

```

        "x-filename": filename,
        "x-filesize": str(size),
        "x-filehash": md5,
        "x-folder-key": params.folder_key,
        "x-mtime": str(int(time.time())),
    }
    async with httpx.AsyncClient(timeout=300.0) as client:
        r = await client.post(
            f"https://www.mediafire.com/api/1.5/upload/simple.php",
            params={"session_token": token, "response_format": "json"},
            headers=headers,
            content=content,
        )
        r.raise_for_status()
        data = r.json().get("response", {})
    if data.get("result") != "Success":
        return f"Upload failed: {data.get('message', 'Unknown error')}}"
    dof = data.get("doupload", {})
    return (f"✅ Uploaded **{filename}** ({_fmt_size(size)}) to folder `{para
        f"- Quick Key: `{dof.get('quickkey', 'pending')}`\n"
        f"- MD5: `{md5}`")
except Exception as e:
    return f"Error uploading: {e}"

if __name__ == "__main__":
    mcp.run()
```